



SQL Injection

I. Khái niệm cơ bản

- SQL Injection là một kỹ thuật cho phép những kẻ tấn công lợi dụng lỗ hổng của việc kiểm tra dữ liệu đầu vào trong các ứng dụng web và các thông báo lỗi của hệ quản trị cơ sở dữ liệu trả về để inject (tiêm vào) và thi hành các câu lệnh SQL bất hợp pháp
- SQL injection có thể cho phép những kẻ tấn công thực hiện các thao tác trên cơ sở dữ liệu của ứng dụng, thậm chí là server mà ứng dụng đó đang chạy.

II. Tính nguy hiểm của SQL injection

- Một cuộc tấn công SQL injection thành công có thể dẫn đến việc truy cập trái phép vào dữ liệu nhạy cảm, chẳng hạn như:
 - + Mật khẩu.
 - + Chi tiết thẻ tín dụng.
 - + Thông tin cá nhân của người dùng.
- Các cuộc tấn công tiêm nhiễm SQL đã được sử dụng trong nhiều vụ vi phạm dữ liệu nghiêm trọng trong nhiều năm qua. Những điều này đã gây ra thiệt hại về danh tiếng và các khoản tiền phạt theo quy định. Trong một số trường hợp, kẻ tấn công có thể có được một cửa hậu liên tục xâm nhập vào hệ thống của tổ chức, dẫn đến sự xâm phạm lâu dài mà có thể không được chú ý trong một thời gian dài.

III. Phân loại

SQL Injection có thể chia nhỏ thành các dạng sau :

- In-band SQLi
 - + Error-based SQLi
 - + Union-based SQLi
- Inferential SQLi (Blind SQLi)
- Blind-boolean-based SQLi
 - + Time-based-blind SQLi



- Out-of-band SQLi

IV. Cách phát hiện lỗ hổng SQL injection

- Ta có thể phát hiện SQL Injection thủ công bằng cách kiểm tra hệ thống đầu vào của ứng dụng theo các bước sau:
 - + Chèn dấu **nháy đơn** ' để kiểm tra lỗi hoặc phản hồi bất thường.
 - + Sử dụng cú pháp SQL để so sánh giá trị đầu vào ban đầu và giá trị thay đổi, rồi so sánh sự khác biệt trong phản hồi.
 - + Điều kiện boolean như **OR 1=1** và **OR 1=2** để xem phản hồi có thay đổi không.
 - + Payload gây trễ thời gian để kiểm tra sự khác biệt về thời gian phản hồi.
 - + OAST payloads để tạo tương tác mạng ngoài băng và giám sát phản hồi.
- Ngoài ra, bạn có thể dùng Burp Scanner để phát hiện lỗ hổng nhanh chóng.

V. Một số ví dụ về SQL injection

- Giả sử một trang web yêu cầu người dùng nhập user_id để lấy thông tin người dùng:
SELECT * FROM users WHERE user_id = '5';
- Trong trường hợp này, nếu người dùng nhập vào **5 OR 1=1**, thì câu truy vấn sẽ trở thành:
SELECT * FROM users WHERE user_id = '5 OR 1=1';
- Do **1=1** **luôn đúng**, nên truy vấn sẽ trả về tất cả các bản ghi trong bảng users, dẫn đến việc rò rỉ thông tin.

VI. Cách ngăn chặn SQL injection

- Ta có thể ngăn chặn hầu hết các cuộc tấn công SQL injection bằng cách sử dụng câu truy vấn có tham số (*prepared statements*) thay vì kết hợp chuỗi.
\$stmt = \$pdo->prepare('SELECT * FROM users WHERE user_id = :user_id');



`$stmt->execute(['user_id' => $user_id]);`

- **Làm sạch (Sanitize)** dữ liệu đầu vào :

Trước khi xử lý dữ liệu từ người dùng, hệ thống cần phải **"làm sạch"** dữ liệu để loại bỏ các ký tự đặc biệt hoặc mã độc có thể gây nguy hại. Ví dụ, loại bỏ các ký tự như ', ", hoặc ; trong các trường nhập liệu từ người dùng.

- **Sử dụng ORM (Object Relational Mapping)**

ORM giúp xây dựng câu truy vấn SQL một cách an toàn bằng cách chuyển đổi các đối tượng trong ngôn ngữ lập trình thành các truy vấn mà không cần nối chuỗi thủ công.

VD : `user = session.query(User).filter(User.id == user_id).one()`

- Câu truy vấn có tham số phù hợp khi dữ liệu không đáng tin cậy được đưa vào trong các điều khoản như **WHERE, INSERT, hoặc UPDATE**. Các phần khác như tên bảng/cột cần được xử lý bằng whitelist hoặc logic đặc biệt. Để hiệu quả, chuỗi trong câu truy vấn phải là hằng số và không chứa dữ liệu biến đổi.

VII. Nguồn tham khảo

- <https://portswigger.net/web-security/sql-injection>
- <https://viblo.asia/p/sql-injection-la-gi-co-bao-nhieu-kieu-tan-cong-sql-injection-m68Z0QnMlkG>
- <https://portswigger.net/blog/oast-out-of-band-application-security-testing>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team